Full-Stack AI/ML Engineer: Technical Assignment

About HireSense.ai

HireSense.ai is a talent intelligence platform that helps organizations understand, find and engage talent through data and AI. It combines large-scale people and company data, search and ranking, machine learning and language models to turn fragmented information into decisions that recruiters can trust.

1. Overview

This assignment is for the Full-Stack AI/ML Engineer position at HireSense.ai. The position combines full-stack engineering, machine learning and applied AI, including LLM-based assistants. The assignment therefore asks for one cohesive product, from the browser to the trained model, rather than separate exercises.

The scope is intentionally open-ended. We estimate that a good submission requires roughly 10 to 12 hours of focused work, and we recommend that you plan your time before you begin. Nobody is expected to complete every item. We assess what you choose to prioritize, how you justify your decisions, and whether your results are measured and reproducible. Please state clearly what you chose not to build, and why.

2. The Product

You will build an **expert-finding product** on a **context graph**. The context graph connects people, papers, venues and collaborations, and every edge carries a date and a source. A user describes a research problem in free text, and the product:

1. returns a ranked list of the people best suited to work on it, with evidence for each result;
2. lets the user refine the search through conversation with an AI assistant that can use the graph as a set of tools and remembers the user's decisions.

All data is public. Use the DBLP computer-science bibliography (https://dblp.org/xml/, released under CC0). You may develop on a smaller slice of it. Document any other public source you use.

DBLP separates authors who share a name with numeric suffixes (for example "Wei Wang 0001" and "Wei Wang 0002"). This provides ground truth for author identity.

3. Ground Rules

- Any language and stack may be used. Python is expected for the machine learning components.
- AI coding tools are allowed. You must be able to explain and defend every part of your submission.
- The system must run offline and without API keys by default. LLM calls are allowed but must be placed behind an interface with a recorded or local fallback. Never commit keys. Keep total API spend below approximately $5.
- All performance figures must come from your own measurements.
- We will run your submission on our own machine against a data slice and queries that you have not seen (see Section 4). Solutions tuned to the development data will be evident.

4. Required Interfaces

4.1 Batch command

```
./run --data <dblp-style-xml-file> --queries <file> --out <file>
```

`--data` is a DBLP-format XML file. A different slice from the one used in development will be supplied (for example a different subfield and time window), so the pipeline must handle any valid DBLP-format file. `--queries` is a text file with one query per line. `--out` is a JSON file containing, for each query, a ranked list of people with a score, an explanation, and the evidence (papers, years, collaborators) behind it. No network access may be required.

4.2 Application

`docker compose up` starts the application: web interface, API and database, plus any additional service you use. The README must give the exact steps to load a small DBLP slice and use the application in five commands or fewer.

5. Technology Context

The technologies below describe our engineering environment. Their use is encouraged but not mandatory. A simpler setup, for example SQLite and an in-process search library in place of a separate search server, is perfectly acceptable. Justify your choices in your design document.

- TypeScript with NestJS for the API; Next.js and React for the web application.
- PostgreSQL as the system of record; Redis for caching and session state.
- Python with FastAPI for the machine learning service.
- Hosted APIs for embeddings and LLM calls.
- Docker containers deployed on AWS.

Our direction includes a deeper context-graph layer, agentic outreach workflows with human approval, a conversational assistant with controllable memory, and learning from user feedback. Your design document should say briefly how your design would accommodate these. They do not need to be implemented.

6. Requirements

Part 1: Data preparation and ground truth

- Parse and clean the DBLP data. Build a time split: papers up to a cutoff year are used to construct the graph, and papers after the cutoff provide ground truth.
- Derive queries and relevance judgments from the held-out period without manual labelling. For example, use the title of a held-out paper as a query and treat its authors as relevant. State what this does and does not measure.
- Create a noisy version of the author data (for example initials in place of first names, name-order swaps and typographical errors), keeping the original DBLP identifiers as hidden ground truth.
- The pipeline must work on at least 50,000 papers.

Part 2: Entity resolution and the context graph

- Resolve authors from the noisy data. Describe your blocking or candidate-generation strategy, because comparing all pairs does not scale.
- Use a matcher that relies on coauthors, venues and years as evidence, and not on names alone. Report precision and recall against the DBLP ground truth, and say briefly which error costs more, an incorrect merge or an incorrect split.
- Build the graph with typed, dated edges (author-paper, author-venue, coauthorship) and store the source of every edge.
- Ingestion must be idempotent: running it twice must not create duplicates.

Part 3: Machine learning

- Build retrieval that combines lexical search with dense vectors. Graph expansion (for example collaborators or venue neighbours) is a worthwhile addition if time permits.

- Train at least one model yourself, for example a fine-tuned embedding model, a cross-encoder reranker or a learned ranking model over features. Justify it, and show its benefit over an off-the-shelf alternative with an ablation.
- Explain how training labels are produced and how you prevent training and evaluating on the same signal.
- Address at least one hard case, for example prolific authors with very broad output, newcomers with little history, or recency.

Part 4: Applied AI

- Query understanding: convert free-text queries into a structured form (topics, recency preference) with an LLM, validated against a schema.
- Grounded explanations: every claim in an explanation must be traceable to a paper or a graph edge. Verify this automatically and report the rate of unsupported claims.
- Conversational assistant: implement a multi-turn assistant that calls at least three tools over your graph (for example search, retrieve profile, and find collaboration path), with arguments validated against schemas. It must keep session memory of shortlist and reject decisions, show the user what memory it applied, and let the user delete it.
- Safety: defend against prompt injection from paper titles and queries, with tests. No tool may be invoked as a result of instructions embedded in data.
- Reliability: a fallback path when the LLM is unavailable.
- Evaluation: define at least 6 scripted multi-turn scenarios with expected tool calls and outcomes, and report task success and injection resistance.

Part 5: Full-stack application

Build the web application with Next.js and React in TypeScript, or an equivalent stack you can justify. It communicates only through your API.

- Search page: a query input and a results list with loading, empty and error states. Stale searches are cancelled.
- Results and evidence: each result shows its score, explanation and expandable evidence. Unsupported claims are visually distinguished.
- Profile page: a publication timeline and topics, with the source of each fact.
- Assistant panel: a chat panel next to the results, in which tool calls and the memory applied are visible to the user.
- Shortlist: shortlist and reject actions that remain correct when a request fails.
- Quality: keyboard navigation, accessible labels and roles, and safe rendering of all model-generated and data-derived text.
- Tests: component tests for key components, and at least one end-to-end test covering search, assistant interaction and shortlist.

Part 6: Operations and quality

- A health endpoint and basic logs.
- Automated tests that run offline and without API keys, including at least one test each for time-split logic, an incorrect merge, a prompt injection and ingestion idempotency.
- A continuous integration configuration that runs the tests.

Part 7: Evaluation

- Report ranking quality (nDCG@10 and MRR) against at least a lexical baseline and an off-the-shelf embedding baseline.
- Provide error analysis of two significant failures, and what you would change.
- Check for leakage between training data, the graph and evaluation data, with attention to the time split.
- Run a simple fairness check, for example swapping name origin or institution while holding everything else equal, and report whether the ranking changes.

Part 8: Design document

Submit a design document of two to three pages covering:

- The data model, graph schema and key decisions, with the alternatives you considered.
- The machine learning design: models, training data, evaluation and serving.
- The assistant design: tools, memory and failure handling.
- How the system would serve 50 million papers and 200 queries per second at a p95 latency of 500 ms, with the main trade-offs.
- Privacy considerations, and how the design would accommodate the directions in Section 5.

7. Deliverables

- A public GitHub repository or an archive containing the code, the data tooling, the tests, the continuous integration configuration and a README with exact run instructions.
- The design document.
- A results file containing your metrics and the commands used to produce them.
- A brief statement of how you used AI tools, including one significant error made by a tool that you identified and corrected.
- A screen recording of three to five minutes demonstrating the running application and the key results.

8. Assessment Criteria

- Machine learning: a trained model, appropriate baselines, an ablation and honest evaluation.
- Applied AI: grounded output, correct tool use, controllable memory and resistance to prompt injection.
- Full-stack engineering: a usable, safe and tested application with a reliable backend.
- Data modelling and correctness: a graph in which each node and edge type has a clear purpose.
- Judgment: sensible scoping and explicit trade-offs.
- Communication: clear documentation and a candid statement of limitations.

9. Submission

Submit by email to navneet@hiresense.ai with the subject line "Full-Stack AI/ML Engineer Assignment - Your Name". Submit before the deadline stated in the message that accompanied this document. Questions may be sent to the same address.

10.1 What to include

1. **Code.** Provide one of the following: - a link to a public repository; or - a single `.zip` archive of the project.
2. **Design document** (PDF). Attach it to the email and also include it in the repository.
3. **Results file** with your metrics and the exact commands that produced them, included in the repository.
4. **AI tools statement** (half a page), included in the repository.
5. **Screen recording** of three to five minutes: attach it if it is under 25 MB, otherwise include a link that does not require sign-in.
6. **Email body**, covering in a few lines: what you implemented, what you omitted, how many hours you spent, and anything we should know before running your project.

10.2 Repository contents

Include all source code, the data tooling, tests and CI configuration, `docker compose` files, a README with exact run instructions (including the batch command in Section 4), small trained model files (under 100 MB in total) or a script that reproduces them, and a `.env.example`. Do not include secrets, the raw DBLP download, dependency folders or build output. If an archive exceeds 25 MB, send a download link that works without sign-in.

Before you send, check that a clean copy of your project starts with the commands in your README, offline and without API keys. A submission that cannot be started will be assessed on the written material only.

10. Next Steps and Confidentiality

We will review your design document and code, and run your system against our own data slice and queries. Shortlisted candidates will be invited to a discussion, in which you present your decisions and results. Submissions are used only for evaluation and will be deleted on request.